Security Advisory

Unauthenticated Network Access Leading to Project File Tampering and Code Execution Risk in Godot-MCP

Advisory Title	Unauthenticated Network Access Leading to Project File Tampering and Code Execution Risk in Godot-MCP
Repository	https://github.com/ee0pdt/Godot-MCP
Commit Tested	b7d08eb
Date Discovered	April 13, 2026
Reported By	Nathaniel Ryan
Contact	nathaniel.ryan.9076@gmail.com
Severity	Critical
CVSS v3.1 Score	9.6 (Critical), updated from 8.8 following runtime confirmation, see Section 2
CVE ID	Pending (requested April 21, 2026, awaiting MITRE assignment)
Disclosure Date	July 14, 2026
Status	Unpatched at publication; no maintainer response received within the disclosure window

Prepared by Nathaniel Ryan
Public disclosure scheduled for July 14, 2026

Table of Contents

1. Executive Summary

A critical vulnerability chain was identified in Godot-MCP, a Godot Engine plugin that integrates AI coding assistants with the Godot editor by exposing editor commands over a local WebSocket server.

Testing in a controlled isolated lab environment confirmed that a combination of missing network binding restrictions and complete absence of authentication allows any device sharing a network with the developer to connect to the WebSocket server and exercise the same privileges granted to the AI agent, including source code exfiltration, silent file modification, and remote code execution on the developer's machine during normal development workflows.

All testing was conducted in isolated virtual environments under researcher control. No external systems or third-party environments were accessed at any point.

2. Vulnerability Details

Affected Components

- `addons/godot_mcp/websocket_server.gd`
- `addons/godot_mcp/commands/script_commands.gd`
- `addons/godot_mcp/commands/editor_script_commands.gd`

Affected Versions

All versions as of commit `b7d08eb`. No patched version was available at the time of research.

CWE Classifications

- **CWE-306:** Missing Authentication for Critical Function
- **CWE-284:** Improper Access Control
- **CWE-94:** Improper Control of Generation of Code ('Code Injection')

CVSS v3.1 Scoring

The vulnerability was originally scored at 8.8 (High) with User Interaction Required, based on the file-injection attack chain requiring the developer to launch the game. Runtime confirmation of the `execute_editor_script` command (see Section 3.7, Chain B) established a second attack vector requiring no user interaction. The score below reflects that updated vector. The original score is retained for reference.

Updated Score (current):

Metric	Value
Attack Vector	Adjacent Network
Attack Complexity	Low
Privileges Required	None
User Interaction	None (updated from Required)
Scope	Changed
Confidentiality Impact	High
Integrity Impact	High
Availability Impact	High

- Base Score: **9.6 (Critical)**
- Temporal Score: **9.4 (Critical)**
- Impact Subscore: 6.0
- Exploitability Subscore: 2.8

Vector String:

```
CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H/E:F/RL:X/RC:C
```

Original Score (first report, April 15):

Metric	Value
Attack Vector	Adjacent Network
Attack Complexity	Low
Privileges Required	None
User Interaction	Required
Scope	Changed
Confidentiality Impact	High
Integrity Impact	High
Availability Impact	High

- Base Score: **8.8 (High)**
- Temporal Score: **8.6 (High)**

Vector String:

```
CVSS:3.1/AV:A/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H/E:F/RL:X/RC:C
```

Both scores calculated via the NIST NVD CVSS v3.1 Calculator.

3. Technical Description

The root cause is an unenforced trust assumption that only authorized local clients will connect to the plugin's WebSocket service. This assumption does not hold in any shared-network environment.

3.1 Network Binding Issue

The plugin initializes a TCP server within the Godot editor using Godot's `TCPServer` class, started with:

```
tcp_server.listen(_port)
```

No bind address is specified. Godot's `TCPServer` defaults to binding on all available network interfaces (`0.0.0.0`) when no address is provided. As a result, the WebSocket server listens on port 9080 on every network interface of the developer's machine, including external interfaces. Any device on the same network as the developer can reach this port directly, without requiring local machine access.

3.2 Missing Authentication

Upon receiving a TCP connection, the server immediately upgrades it to a WebSocket connection and adds the client to the active peers list without any authentication or authorization check. The connection flow is:

1. TCP connection received
2. WebSocket handshake completed
3. Client added to peers dictionary
4. Full command access granted

No token, credential, origin check, or any other verification mechanism exists at any point in this flow. Any connecting client receives immediate access to all available commands regardless of identity or authorization.

3.3 Unauthenticated File Read

The plugin exposes multiple commands for reading script files and mapping the project structure:

- `get_current_script` returns the full source code of whatever script is currently open in the Godot editor
- `get_script` returns the full source code of any script file within the project directory when provided with a file path
- `list_project_resources` returns a complete map of the project directory including paths of all scripts, scenes, textures, audio files, and other resources

None of these commands performs any verification of the requesting client's identity or authorization level. An unauthenticated attacker can first enumerate all script files via `list_project_resources`, then systematically call `get_script` on each returned path to retrieve complete source code.

This enables targeted credential harvesting across an entire project without any prior knowledge of file names or paths. Hardcoded API keys, database credentials, authentication tokens, and any other sensitive information embedded in scripts are fully exposed through this attack chain.

3.4 Unauthenticated File Write

The plugin exposes the `edit_script` command, which accepts a file path and content string and overwrites the target file with the provided content. No authentication is required. Any connecting client can overwrite any script file within the project directory with arbitrary content.

The command executes silently from the perspective of the developer. The Godot editor logs the operation in its output console, but this output may go unnoticed during active development workflows, particularly when AI assistant output is also being logged to the same console and developers have become habituated to ignoring routine log lines.

3.5 Remote Code Execution via Persistence Annotation

GDScript supports a `@tool` annotation which instructs the Godot engine to execute the annotated script within the editor context. By combining the unauthenticated file write capability with this annotation, an attacker can inject malicious GDScript that executes when the developer next launches their game.

The injected code runs with the same OS-level permissions as the Godot editor process.

Testing confirmed successful execution of arbitrary OS commands via `OS.execute()` embedded in an injected `@tool` script. A benign payload (launching the Windows calculator) was used to demonstrate code execution. Filesystem write access beyond the Godot project directory was also confirmed through creation of a file in the victim user's home directory.

Subsequent testing confirmed that execution triggers on Godot editor restart as well as on game launch. A `@tool`-annotated script attached to a scene node executes on any subsequent opening of the editor with that project loaded. Game launch is not required. Node attachment remains a required precondition, a `@tool` script not attached to any scene node does not execute on editor restart. This means the payload fires on whichever comes first, the developer's next game launch or the next time the developer opens the project in the editor. Both are normal development actions requiring no social engineering. See Appendix A.2.

3.6 Secondary Finding: execute_editor_script

The plugin exposes an `execute_editor_script` command which, based on code review, appears to accept arbitrary GDScript code for execution within the editor context. This would represent a more direct unauthenticated code execution path than the file-injection workflow demonstrated above, bypassing the `@tool` annotation and game-launch precondition entirely.

This command was identified through static code review but was **not fully validated during runtime testing**. It should be treated as a high-risk additional finding pending independent confirmation.

This finding was subsequently confirmed through runtime testing. See Appendix A.2.

3.7 Confirmed Attack Chains

Runtime testing confirmed two distinct attack chains. Both begin with an unauthenticated connection from a network-adjacent device and both result in code execution with the permissions of the Godot editor process.

Chain A — File Injection. The attacker connects unauthenticated, calls `list_project_resources` to retrieve the full project file map, identifies a script already attached to a scene node, and calls `edit_script` to inject a `@tool` payload containing `OS.execute()`. Execution triggers on the developer's next game launch or on the next Godot editor restart with the project loaded, whichever comes first. No developer interaction with the targeted script is required at any stage.

Chain B — Direct Execution. The attacker connects unauthenticated, receives the WebSocket welcome message, and sends an `execute_editor_script` command carrying an arbitrary GDScript payload. The payload executes immediately. No file modification, no node attachment, no game launch, and no developer action of any kind is required. The only precondition is an active Godot editor session with the plugin enabled. This chain is the basis for the User Interaction: None metric in the updated CVSS score.

4. Proof of Concept

All testing was conducted in an isolated two-VM lab under Hyper-V, with both VMs on an internal-only virtual switch. Neither VM had connectivity to external networks or any production systems.

4.1 Test Environment

- **Victim Machine:** Windows 11 Development Environment (Hyper-V VM)
- **Godot Version:** 4.5 stable
- **Plugin Version:** Godot-MCP commit `b7d08eb`
- **Attacker Machine:** Ubuntu 22.04 LTS (Hyper-V VM)
- **Network:** Internal-only Hyper-V virtual switch

4.2 Attack Preconditions

- Godot editor is running on the victim machine
- Godot-MCP plugin is enabled in the victim's project
- Attacker has network access to the same subnet as the developer

4.3 Tests Conducted

Seven initial tests were performed during the first validation phase. Additional follow-on tests were later conducted to validate editor restart behavior, direct execute_editor_script execution, and exported build persistence.

Test 1 — Connection without Godot Running. Confirmed that the WebSocket server only runs when Godot is open with the plugin active. Attack surface is bounded to active development sessions. Connection to port 9080 on the victim VM timed out when Godot was not running.

Test 2 — Unauthenticated Network Connection. Confirmed that any device on the same network can establish a WebSocket connection without credentials. Connection succeeded from the attacker VM with no authentication challenge.

Test 3 — Source Code and Credential Exfiltration. Confirmed that `get_current_script` returns the full source of whatever script is open in the editor, and that this source can include hardcoded credentials embedded during normal development. A test script containing a hardcoded API key was successfully retrieved from the victim.

Test 4 — Remote File Modification. Confirmed that `edit_script` can overwrite any project script with attacker-controlled content. Modification was silent from the attacker's perspective; the Godot editor prompted a file reload on the victim side, but this prompt is routine during development and often dismissed without scrutiny.

Test 5 — Path Traversal Outside Project Directory. Confirmed that Godot's virtual filesystem (`res://`) sandboxes file operations within the project directory. Attempts to navigate outside via `../` sequences returned errors. Attack surface for file operations is bounded to the project directory.

Test 6 — Project Directory Reconnaissance. Confirmed that `list_project_resources` returns a complete map of the project directory including all script, scene, and resource paths. This enables targeted follow-on attacks against specific files without prior knowledge of the project structure.

Test 7 — Remote Code Execution. Confirmed the full attack chain: attacker connects unauthenticated, enumerates scripts, overwrites a script with a `@tool`-annotated payload containing `OS.execute()`, developer launches game, payload executes with editor privileges. A calculator was launched on the victim machine as proof of OS-level command execution, and a file was created at `C:\Users\User\pwned.txt` confirming filesystem write access beyond the Godot project directory.

4.4 Evidence

Screenshots documenting successful exploitation (calculator launched on victim, modified file contents visible in editor, file created in user directory) are retained as evidence and are available for verification by CERT teams or the MITRE CVE Numbering Authority upon request. Working exploit scripts are **not included in this advisory** to reduce risk of opportunistic exploitation by unsophisticated attackers.

5. Impact

5.1 Immediate Impact on Developer

An unauthenticated attacker with network access to the developer's machine can achieve the following without any credentials or prior knowledge of the project:

- Full source code exfiltration of all project scripts including hardcoded API keys, authentication tokens, database credentials, and other sensitive information
- Silent modification of any project script without the developer's active awareness
- Remote code execution on the developer's machine with the OS-level permissions of the Godot editor process, either immediately through execute_editor_script or persistently through injection of @tool-annotated GDScript into project files.
- Complete project structure reconnaissance enabling targeted follow-on attacks

5.2 Supply Chain Impact

The most severe long-term impact is the potential for supply chain attacks against end users. An attacker who successfully injects malicious code into a game project's scripts may cause that code to be distributed to all players who download and run the published game. Depending on the nature of the injected payload, this may affect downstream end users if malicious code introduced through this vulnerability is included in distributed game builds, potentially impacting all users of affected applications.

This risk was subsequently confirmed at runtime. See Section 5.5 for the confirmed export behaviour, including the finding that an injected payload persists into exported builds even when the addon is excluded from the export.

5.3 Target Population Risk

The Godot-MCP plugin is designed for integration with AI coding assistants and is primarily adopted by developers using AI-assisted game development workflows. This population may reasonably expect frequent automated tool activity, external assistant connections, and routine file modifications during AI-assisted development workflows. As a result, unauthorized activity may be harder to distinguish from normal assistant-driven behavior.

This significantly increases the realistic exploitability of the vulnerability beyond what the CVSS score alone reflects.

5.4 Broader Ecosystem Exposure

At the time of research, the original repository had 527 stars and 56 forks, indicating active adoption across the developer community. The vulnerability affected all versions of the plugin with no patched version available in the upstream repository at disclosure.

Developers who have forked the repository may also be running vulnerable code. One branch was identified during research that partially addressed the network binding issue by restricting the server to localhost, however the authentication gap and unrestricted file access remain present across all known versions at the time of research.

5.5 Confirmed Supply Chain Persistence

Follow-on testing confirmed the supply chain risk identified in 5.2 and clarified the mechanism behind it. Two separate findings apply to exported builds.

First, the WebSocket server does not initialise in exported builds. Export testing confirmed that a compiled game does not open port 9080 and does not start the WebSocket server when run outside the Godot editor. End users who download and run an exported game are not exposed to the network attack vector described in this advisory. The attack surface for the WebSocket vulnerability is limited to active Godot editor sessions on the developer's machine.

Second, and separately, a malicious payload injected into a project script before export persists into the compiled build. Once code has been written to a project file via `edit_script`, it becomes part of the project source and is included in any subsequent export. Testing confirmed that a game exported with the addons folder explicitly excluded still contained and executed the injected `OS.execute()` payload when run on a clean victim machine with no Godot Editor installed. The plugin's presence is only required at the time of injection. After that point the payload persists independently of the plugin.

The practical consequence is that an attacker does not need the plugin to remain installed for their payload to reach end users. The damage is done at injection time. Any game built from a project that was compromised while the plugin was active may carry the payload to every end user who downloads and runs it, regardless of whether those users have Godot installed and regardless of whether the addon was excluded from the export.

6. Remediation

6.1 Bind Server to Localhost Only (High Priority)

The WebSocket server should be restricted to accept connections from localhost only by specifying a bind address in the listen call:

```
tcp_server.listen(_port, "127.0.0.1")
```

This prevents network-adjacent attackers from reaching the server and significantly reduces the attack surface. A community fork of the repository has already implemented this fix, confirming it is a straightforward and effective mitigation.

6.2 Implement Token Authentication (High Priority)

A shared secret token should be generated on server startup and required from all connecting clients during the WebSocket handshake. Connections that do not present a valid token should be rejected immediately. The token should be included in the MCP client configuration and rotated on each server restart.

This ensures that even if the server is reachable, only authorized clients with knowledge of the current token can execute commands.

6.3 Restrict Sensitive Commands (Medium Priority)

Commands that read or write files and execute code should be restricted to authenticated sessions only. The principle of least privilege should be applied: grant only the permissions necessary for the intended AI agent workflow. This reduces the potential impact of any future authentication bypass.

6.4 Implement Command Allowlist (Medium Priority)

Rather than exposing all available commands to any connecting client, consider implementing an explicit allowlist of permitted commands per session. This limits the attack surface even in scenarios where authentication is bypassed or credentials are compromised.

6.5 Validate and Sanitize File Paths (Medium Priority)

While testing confirmed that Godot's virtual filesystem prevents path traversal outside the project directory, this protection relies on engine behavior rather than explicit validation in the plugin code. Explicit rejection of paths containing `../` sequences should be added as a defense-in-depth measure.

6.6 User Notification (Low Priority)

The plugin currently logs new connections to the Godot output console. Consider adding a more prominent visual notification, such as a dialog or status panel indicator, when a new client connects. This gives developers a clearer signal that an external connection has been established.

6.7 Restrict or Remove execute_editor_script (Critical Priority)

The `execute_editor_script` command should be removed from the plugin entirely, or restricted to explicitly authenticated and authorized sessions only. This command provides direct arbitrary code execution in the editor context with no authentication boundary in the current implementation. Runtime testing confirmed it as a direct unauthenticated remote code execution vector requiring no preconditions beyond an active editor session, with no file modification, node attachment, or game launch needed. Until authentication is implemented, this command represents an unmitigated critical remote code execution vector and should be treated as the highest remediation priority.

7. Recommendations for Users

If you use Godot-MCP in your development environment:

Immediate mitigation:

- **Do not run Godot-MCP on untrusted networks.** This includes coffee shops, coworking spaces, shared WiFi, conference networks, and any network where unknown devices may be present. The vulnerability is exploitable by any device that can reach your machine on port 9080.
- **Consider using the community fork that binds to localhost**, or apply the single-line patch yourself locally.
- **Do not use the plugin on a machine with hardcoded credentials in project scripts.** Any credentials in your project's GDScript files are readable by any network-adjacent attacker while the editor is running with the plugin enabled.

Longer-term mitigation:

- Audit your Godot projects for any hardcoded credentials that may have been exposed while the plugin was running on a shared network
- Consider disabling the plugin until a patched version is released
- Monitor the upstream repository and authoritative community forks for patches

8. Timeline

Date	Event
April 13, 2026	Vulnerability identified through code review of repository at commit `b7d08eb`
April 13-14, 2026	Initial testing conducted in controlled multi-VM lab environment
April 15, 2026	CVSS score calculated and initial advisory drafted; maintainer contacted via public GitHub issue requesting private disclosure channel
April 20, 2026	Additional testing conducted, Tests 07.1, A.1, A.2, and A.3 confirmed
April 20, 2026	Export build testing conducted, supply chain persistence confirmed, Section 5.5 resolved
April 20, 2026	Findings incorporated, CVSS score updated to reflect the zero-interaction vector
April 21, 2026	CVE ID requested
May 18, 2026	Second contact via public GitHub issue requesting private disclosure channel
June 10, 2026	Maintainer contacted via LinkedIn message requesting private disclosure channel, and informing of the public disclosure date
July 14, 2026	Coordinated disclosure deadline; public advisory released

9. Disclosure Statement

This research was conducted under a 90-day coordinated disclosure policy, consistent with industry standards (CERT/CC, Google Project Zero, and most established vulnerability research programs). The maintainer was contacted on April 15, 2026, with a request for a private disclosure channel.

The maintainer did not respond to the initial contact or to follow-up attempts within the disclosure window. This advisory is published to inform the community of an unpatched vulnerability in a publicly-available tool so that affected users can apply mitigations.

No working exploit code is published with this advisory. The technical detail provided is intended to enable defenders, downstream security tooling, and package auditing systems to identify and mitigate the vulnerability. It is not intended to lower the barrier to exploitation.

10. Credits

Researcher: Nathaniel Ryan

Contact: nathaniel.ryan.9076@gmail.com

Discovery Date: April 13, 2026

Disclosure Method: Coordinated disclosure, 90-day timeline, maintainer contacted privately via GitHub issue prior to public publication.

Tools Used

- Python 3.10.4 with `websocket-client` 1.9.0 (proof-of-concept scripts)
- Godot Engine 4.5 stable (test target)
- Hyper-V virtual machine environment (isolated lab)
- NIST NVD CVSS v3.1 Calculator (severity scoring)
- MITRE CWE database (classification)

Appendix A: Further Research

The following research questions were identified during the initial engagement and investigated in the period between discovery and disclosure. Results are included here where available.

A.1 Exported Game Build Behavior

Question: Does the WebSocket server initialize when a game is exported and run outside the Godot editor?

Findings: No the WebSocket server is only enabled during live Godot Editor Sessions.

A.2 execute_editor_script Direct RCE Vector

Question: Does the `execute_editor_script` command accept and execute arbitrary GDScript without the `@tool` + game-launch precondition?

Findings: Yes it is possible to achieve RCE outside the project environment with only an active Godot Editor session.

A.3 Non-Active Script Targeting

Question: Can `edit_script` silently modify scripts already attached to scene nodes, such that malicious code executes on next game launch without any additional setup by the developer?

Findings: Yes. Testing confirmed that a malicious payload injected into a project script before export can persist into the exported game. The exported game executed the payload on a clean victim VM with no Godot Editor installed and no WebSocket connection present.

This advisory is published for defensive purposes. The information contained is intended to enable users, maintainers, and security tooling to identify and mitigate the vulnerability. It is not intended to enable or facilitate attacks against any system. Unauthorized testing of this vulnerability against systems not owned or controlled by the tester may violate computer fraud laws in many jurisdictions.